

# Assignment-19.1

**Name:** P.Karthik

**Hall.No:** 2303A51908

**Batch – 12**

## **Task 1 – Python to Java Conversion**

Task:

Translate a simple Python class into Java using AI assistance.

Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

CODE:

```
1 class Student:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def display(self):
7         print("Name:", self.name)
8         print("Age:", self.age)
9
10 s = Student("Akram", 21)
11 s.display()
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```

```
1 class Student {
2     Student(String name, int age) {
3         this.name = name;
4         this.age = age;
5     }
6
7     // Method
8     void display() {
9         System.out.println("Name: " + name);
10        System.out.println("Age: " + age);
11    }
12
13    Run | Debug
14    public static void main(String[] args) {
15        Student s = new Student(name: "Akram", age: 21);
16        s.display();
17    }
18 }
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & 'C:\Program Files\Java\jdk-17\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\akram\AppData\Roaming\Code\User\workspaceStorage\c81bc8234e12757088efddabec550e35\redhat_java\jdt_ws\AIAC_26af4a4e\bin' 'Student'
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```

## Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.

- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

CODE:

```

task2.py
1 def factorial(n):
2     if n == 0:
3         return 1
4     else:
5         return n * factorial(n-1)
6
7 print(factorial(5))

```

```

PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Name: Akram
Age: 21
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC>

```

```

task_2.cpp
1 #include <iostream>
2 using namespace std;
3
4 int factorial(int n) {
5     if(n == 0)
6         return 1;
7     else
8         return n * factorial(n-1);
9 }
10
11 int main() {
12     cout << factorial(5);
13     return 0;
14 }

```

```

PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Name: Akram
Age: 21
120
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC>

```

## Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., `SELECT name, salary FROM employees WHERE salary > 50000`).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

CODE:

The screenshot shows the Visual Studio Code editor with a Python file named `task_3.py` open. The code defines a dictionary `data` with names and salaries, creates a `pandas` `DataFrame`, and filters for salaries greater than 50,000. The terminal shows the command `python task_3.py` being executed, resulting in the following output:

```
Age: 21
name salary
1 Alice 60000
2 Bob 75000
```

The screenshot shows the Visual Studio Code editor with a SQL file named `task_3.sql` open. The code contains a SQL query to select names and salaries from an `employees` table where the salary is greater than 50,000. The terminal shows the command `python task_3.py` being executed, resulting in the following output:

```
name salary
1 Alice 60000
2 Bob 75000
```

## Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.

- Translating a Java method into C# for enterprise applications.

Instructions:

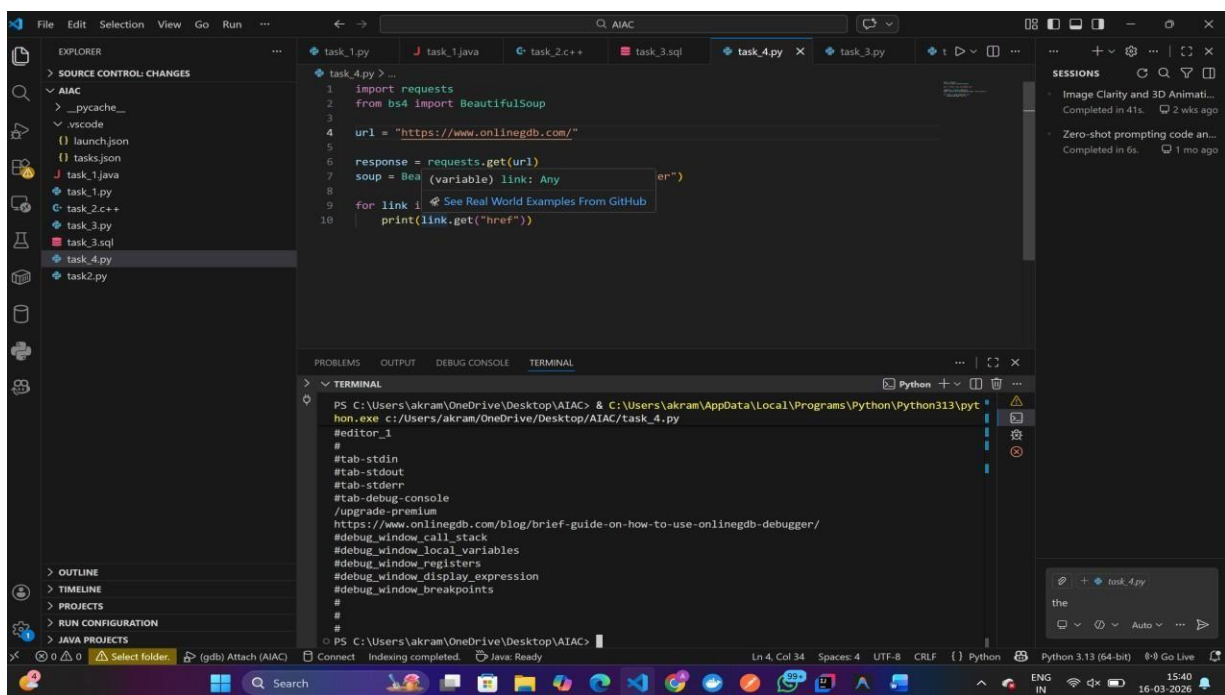
- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.

- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages

CODE:



The screenshot shows a Visual Studio Code editor window with a dark theme. The Explorer sidebar on the left shows a project named 'AIAC' with files like 'task\_1.py', 'task\_2.c++', 'task\_3.sql', 'task\_4.py', and 'task2.py'. The main editor area displays the content of 'task\_4.py', which is a Python script using the 'requests' and 'BeautifulSoup' libraries to fetch data from 'https://www.onlinegdb.com/'. The script includes comments and a loop to print href values. The bottom panel shows the 'TERMINAL' tab with a command prompt where the script is executed using 'python.exe'. The terminal output shows the script running successfully, printing the href values. The status bar at the bottom indicates the file is 'task\_4.py' and the Python interpreter is 'Python 3.13 (64-bit)'.

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 url = "https://www.onlinegdb.com/"
5
6 response = requests.get(url)
7 soup = BeautifulSoup(response.text, 'html.parser')
8
9 for link in soup.find_all('a'):
10     print(link.get("href"))
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_4.py
#
#tab-stdin
#tab-stdout
#tab-stderr
#tab-debug-console
/upgrade-premium
https://www.onlinegdb.com/blog/brief-guide-on-how-to-use-onlinegdb-debugger/
#debug_window_call_stack
#debug_window_local_variables
#debug_window_registers
#debug_window_display_expression
#debug_window_breakpoints
#
#
```

